Ministerul Educației și Cercetării al Republicii Moldova

Universitatea Tehnică a Moldovei

Facultatea Calculatoare, Informatică și Microelectronică

Departamentul Ingineria Software și Automatică

Laboratory Work 3:

The Command Line: Shell, Pipes and Text Filters

Elaborated: st. gr. FAF-243

Șoimu Ionuț

Verified: asist. univ.

Reițman Patricia

Chișinău – 2026

Contents

1 Lab 3: Track A – The Command Line: Shell, Pipes and Text Filters **2**
1.1 Before you start . . . 2
1.2 Part 1. Move fast, find help . . . 2
1.2.1 1. Tab Completion . . . 2
1.2.2 2. Command History and Reverse Incremental Search . . . 2
1.2.3 3. Built-ins vs. External Commands, Search Path and Exit Codes . . . 3
1.2.4 4. Built-in Help and Man Pages . . . 3
1.3 Part 2. What the shell does to your line . . . 4
1.3.1 Test Directory Setup . . . 4
1.3.2 Prediction vs. Real Output Table . . . 5
1.3.3 Terminal Execution Record . . . 6
1.4 Part 3. Redirection, chaining and jobs . . . 7
1.4.1 Redirection and Chaining Executions . . . 7
1.4.2 Comparing Redirection Ordering . . . 7
1.5 Part 4. Answer questions with one-liners . . . 8
1.6 Part 5. Your first script . . . 11
1.6.1 Completed report.sh Script . . . 11
1.6.2 Execution and Validation . . . 11

2 Conclusion **12**

1 Lab 3: Track A – The Command Line: Shell, Pipes and Text Filters

1.1 Before you start

Purpose and Scope

The objective of this laboratory work is to develop deep command-line proficiency within the Linux operating system, master the internal expansion and substitution mechanisms of the Bourne Again Shell (`bash`), understand I/O stream redirection and job management, and harness the Unix philosophy of constructing modular data-processing pipelines using core text filters joined by pipes. Finally, common analytical one-liners are compiled into an automated, production-grade shell script.

Environment Setup and Identification

```
➜  ~ mkdir -p ~/os-lab3 && cd ~/os-lab3
➜  os-lab3 whoami
ionut
➜  os-lab3 uname -a
Linux arch 7.2.8-1-cachyos #1 SMP PREEMPT_DYNAMIC Sat, 26 Sep 2026 19:43:45 +0000
    x86_64 GNU/Linux
➜  os-lab3 uptime
 17:34:08 up  3:56,  1 user,  load average: 0.42, 0.48, 0.51
```

Data Set Generation

We generate the standardized test dataset representing a web server access log of 500 entries:

```
➜  os-lab3 for i in $(seq 1 500); do
echo "2026-10-05 10:$(printf %02d $((i % 60))) user$((i % 7)) $([ $((i % 5)) -eq 0 ]
    && echo FAIL || echo OK) /page$((i % 4))"
done > access.log
➜  os-lab3 head -3 access.log ; wc -l access.log
2026-10-05 10:01 user1 OK /page1
2026-10-05 10:02 user2 OK /page2
2026-10-05 10:03 user3 OK /page3
500 access.log
```

The verification command confirms that `access.log` contains exactly 500 lines.

1.2 Part 1. Move fast, find help

1.2.1 1. Tab Completion

When typing `ls /usr/sh` and pressing the `Tab` key, the shell scans matching files and directories starting with `sh` in `/usr`. In our system, it immediately auto-completes to `/usr/share/`. When typing `cd /et` and pressing `Tab`, the shell recognizes that `/etc` is the unique directory matching the prefix, completing the line to `cd /etc/`. Tab completion drastically reduces typographical errors and accelerates path navigation.

1.2.2 2. Command History and Reverse Incremental Search

After executing multiple commands, pressing `Ctrl+R` triggers bash's reverse incremental search:

```
(reverse-i-search)‘seq’: for i in $(seq 1 500); do echo "2026-10-05 10:$(printf %02d
    $((i % 60))) user$((i % 7)) $([ $((i % 5)) -eq 0 ] && echo FAIL || echo OK) /
    page$((i % 4))"; done > access.log
```

Typing a few characters immediately searches backward through the history buffer, allowing previously entered complex commands to be recalled and executed without re-typing.

1.2.3 3. Built-ins vs. External Commands, Search Path and Exit Codes

```
➜  os-lab3 type cd ; type ls ; type echo ; which ls
cd is a shell builtin
ls is /usr/bin/ls
echo is a shell builtin
/usr/bin/ls
➜  os-lab3 echo $PATH
/usr/local/sbin:/usr/local/bin:/usr/bin:/var/lib/flatpak/exports/bin:/usr/lib/jvm/
    default/bin:/usr/bin/site_perl:/usr/bin/vendor_perl:/usr/bin/core_perl:/opt/rocm/
    bin
➜  os-lab3 echo $PATH | tr ’:’ ’\n’
/usr/local/sbin
/usr/local/bin
/usr/bin
/var/lib/flatpak/exports/bin
/usr/lib/jvm/default/bin
/usr/bin/site_perl
/usr/bin/vendor_perl
/usr/bin/core_perl
/opt/rocm/bin
➜  os-lab3 nosuchprog ; echo "exit code: $?"
bash: line 1: nosuchprog: command not found
exit code: 127
```

1.2.4 4. Built-in Help and Man Pages

```
➜  os-lab3 man ls
# inside: /sort then n for next, q to quit
➜  os-lab3 ls --help | head -5
Usage: ls [OPTION]... [FILE]...
List information about the FILEs (the current directory by default).
Sort entries alphabetically if none of -cftuvSUX nor --sort is specified.

Mandatory arguments to long options are mandatory for short options too.
➜  os-lab3 man -k "list directory"
dir (1)              - list directory contents
ls (1)               - list directory contents
ntfsls (8)           - list directory contents on an NTFS filesystem
vdir (1)             - list directory contents
➜  os-lab3 help cd | head -4
cd: cd [-L|[-P [-e]] [-@]] [dir]
    Change the shell working directory.

    Change the current directory to DIR.  The default DIR is the value of the
    HOME shell variable.
```

Observe:

1. **Which of cd, ls, echo are built-ins, and why MUST cd be one?**
   `cd` and `echo` are shell built-ins, while `ls` is an external binary located at `/usr/bin/ls`. `cd` **must** be a built-in because in Unix-like operating systems, the current working directory is an attribute of the process itself (stored in the process descriptor in kernel space). When an external program is executed, the shell forks a child process and runs the program in that child. If `cd` were an external binary, the child process would change its own directory and then terminate; the parent shell's working directory would remain unchanged. Hence, `cd` must execute directly inside the parent shell process.

2. **How many directories are in your PATH, and in which one is ls?**
   There are **10 directories** listed in `$PATH`. The binary for `ls` resides in `/usr/bin` (specifically `/usr/bin/ls`).

3. **What exit code does a command that is not found return?**
   The shell returns exit code **127** when an attempted command cannot be found within any directory listed in `$PATH`.

4. **Which option of ls sorts by modification time? (find it in man ls)**
   The option is `-t`, which instructs `ls` to sort entries by modification time, newest first.

1.3 Part 2. What the shell does to your line

1.3.1 Test Directory Setup

```
➜  os-lab3 mkdir -p ~/os-lab3/play && cd ~/os-lab3/play
➜  play touch a.c b.c file1.txt file2.txt notes.txt "alfa beta.txt"
➜  play ls
 a.c  'alfa beta.txt'   b.c   file1.txt   file2.txt   notes.txt
```

1.3.2 Prediction vs. Real Output Table

Command	My Prediction	Real Output	Explanation (if different)
`echo *.c`	`a.c b.c`	`a.c b.c`	Glob expansion matched all files ending in `.c`.
`echo file?.txt`	`file1.txt file2.txt`	`file1.txt file2.txt`	Wildcard `?` matched single character `1` and `2`.
`echo [a-b]*`	`a.c b.c`	`a.c alfa beta.txt b.c`	Character class `[a-b]` also matched `alfa beta.txt` since it starts with `a`.
`echo *.xyz`	empty / error	`*.xyz`	In bash without `nullglob`, an unmatched pattern remains as a literal string.
`ls alfa beta.txt`	`alfa beta.txt`	Error: cannot access	Unquoted space caused bash to split into two arguments: `alfa` and `beta.txt`.
`ls "alfa beta.txt"`	`'alfa beta.txt'`	`'alfa beta.txt'`	Double quotes preserved whitespace as a single argument.
`echo '$HOME' "$HOME" \$HOME`	`$HOME /home/ionut $HOME`	`$HOME /home/ionut $HOME`	Single quotes inhibit expansion; double quotes expand; backslash escapes.
`echo {1..5} ; echo $((7 * 6))`	`1 2 3 4 5 42`	`1 2 3 4 5 42`	Brace expansion generated sequence; `$(())` evaluated arithmetic expression.
`echo "today is $(date +%A), I am $(whoami)"`	Current weekday and username	`today is Monday, I am ionut`	Command substitution `$(...)` ran nested commands and inserted outputs.
`touch log{1..3}.txt && ls log*`	`log1.txt log2.txt log3.txt`	`log1.txt log2.txt log3.txt`	Brace expansion created 3 files; globbing matched and listed them.

Table 1: Command-line expansion predictions and empirical results

1.3.3 Terminal Execution Record

```
➜  play echo *.c
a.c b.c
➜  play echo file?.txt
file1.txt file2.txt
➜  play echo [a-b]*
a.c alfa beta.txt b.c
➜  play echo *.xyz
*.xyz
➜  play ls alfa beta.txt
ls: cannot access 'alfa': No such file or directory
ls: cannot access 'beta.txt': No such file or directory
➜  play ls "alfa beta.txt"
'alfa beta.txt'
➜  play echo '$HOME' "$HOME" \$HOME
$HOME /home/ionut $HOME
➜  play echo {1..5} ; echo $((7 * 6))
1 2 3 4 5
42
➜  play echo "today is $(date +%A), I am $(whoami)"
today is Monday, I am ionut
➜  play touch log{1..3}.txt && ls log*
log1.txt  log2.txt  log3.txt
```

Observe:

1. **For each line: your prediction, the real output, and one sentence if they differ.**
   Refer to Table 1 above. The primary differences occurred with unmatched patterns (`*.xyz`) where bash defaults to preserving the literal pattern rather than returning null, and with `[a-b]*` which matched `alfa beta.txt` as well.

2. **echo knows nothing about files, yet echo *.c printed file names. Who produced them?**
   The shell (`bash`) produced them during its **path expansion (globbing)** phase. Before the `echo` process is invoked, bash inspects the command line, detects the unquoted wildcard `*.c`, scans the current working directory for matching filesystem entries, replaces `*.c` with `a.c b.c`, and passes these resolved strings as separate entries in `argv` to `echo`.

3. **Why did ls alfa beta.txt fail? Give two different ways to fix it.**
   The shell performs **word splitting** on unquoted whitespace. Because `alfa beta.txt` was unquoted, bash split the string at the space into two separate arguments: `alfa` and `beta.txt`. `ls` then tried to locate both files, neither of which exists individually.
   Two ways to fix it:
   - Quoting the string: `ls "alfa beta.txt"` (or single quotes `ls 'alfa beta.txt'`)
   - Escaping the space character: `ls alfa\ beta.txt`

4. **What is the difference between single and double quotes? Why does ls show the file with a space as 'alfa beta.txt'?**
   Single quotes (`'...'`) preserve the literal value of each character within the quotes; no expansions (parameter, arithmetic, command substitution, or globbing) occur.
   Double quotes (`"..."`) suppress globbing and word splitting while still allowing parameter expansion (`$VAR`), command substitution (`$(...)`), and arithmetic expansion (`$((...))`).

GNU `ls` quotes filenames containing spaces or non-printable characters as `'alfa beta.txt'` using shell quoting syntax so that users can clearly disambiguate a single file name with internal whitespace from multiple adjacent files.

1.4 Part 3. Redirection, chaining and jobs

1.4.1 Redirection and Chaining Executions

```
➜  play cd ~/os-lab3
➜  os-lab3 ls /etc /nope > out.txt 2> err.txt
➜  os-lab3 wc -l out.txt ; cat err.txt
215 out.txt
ls: cannot access '/nope': No such file or directory
➜  os-lab3 ls /etc /nope > all.txt 2>&1 ; tail -2 all.txt
xml
zsh
➜  os-lab3 head -2 all.txt
ls: cannot access '/nope': No such file or directory
/etc:
➜  os-lab3 echo first > notes.md ; echo second >> notes.md ; cat notes.md
first
second
➜  os-lab3 wc -l < access.log
500
➜  os-lab3 ls /nope 2> /dev/null || echo "failed with code $?"
failed with code 2
➜  os-lab3 mkdir -p tmp && cd tmp && echo "I am in $(pwd)" ; cd ~/os-lab3
I am in /home/ionut/os-lab3/tmp
➜  os-lab3 sleep 100 &
[1] 23150
➜  os-lab3 jobs ; kill %1 ; sleep 0.1 ; jobs
[1]+  Running                 sleep 100 &
[1]+  Terminated              sleep 100
```

1.4.2 Comparing Redirection Ordering

```
# Version A:
➜  os-lab3 ls /etc /nope > all.txt 2>&1
# (Both directory listing and error message are written into all.txt)

# Version B:
➜  os-lab3 ls /etc /nope 2>&1 > all2.txt
ls: cannot access '/nope': No such file or directory
# (Error message is displayed on terminal; only directory listing is written into
    all2.txt)
```

Observe:

1. **Which file got the error message, and which got the listing?**

   `err.txt` received the error message (`ls: cannot access '/nope'...`), while `out.txt` received the directory listing of `/etc`.

2. **What is the difference between ¿ all.txt 2¿&1 and 2¿&1 ¿ all.txt? Try both and explain why the result differs.**

Redirections in bash are processed strictly from **left to right**:

- In `> all.txt 2>&1`: File descriptor 1 (stdout) is first redirected to open file `all.txt`. Then, `2>&1` duplicates file descriptor 1 onto file descriptor 2 (stderr). Both descriptors now refer to `all.txt`, so standard output and errors are captured together.
- In `2>&1 > all.txt`: First, descriptor 2 (stderr) is duplicated to wherever descriptor 1 currently points (the terminal). Next, descriptor 1 is redirected to `all.txt`. Descriptor 2 continues pointing to the terminal, so error messages are printed directly to the screen while standard output is routed to the file.

3. **Why does wc -l ¡ access.log print only a number, while wc -l access.log also prints the file name?**

   When invoked as `wc -l access.log`, the filename is passed as a command-line argument. `wc` opens the file itself, counts its lines, and prints the filename alongside the total count. When invoked with `wc -l < access.log`, the shell redirects the file into standard input (fd 0) before executing `wc`. `wc` receives no filename arguments and simply reads from anonymous stream 0, having no knowledge of the underlying file's path.

4. **Explain each operator in one sentence: ; && —— .**

   - `;` executes the preceding command and immediately proceeds to the next command unconditionally, regardless of exit status.
   - `&&` executes the subsequent command only if the preceding command succeeded (returned exit code 0).
   - `||` executes the subsequent command only if the preceding command failed (returned a non-zero exit code).

1.5 Part 4. Answer questions with one-liners

The generated `access.log` file adheres to a structured, space-delimited schema:

`[Date] [Time] [User] [Status] [Page]`

Q1. How many requests failed?

```
➜  os-lab3 grep -c 'FAIL' access.log
100
```

Result: 100 requests failed.

Q2. How many different pages were requested, and which?

```
➜  os-lab3 cut -d' ' -f5 access.log | sort -u
/page0
/page1
/page2
/page3
```

Result: 4 different pages were requested: `/page0`, `/page1`, `/page2`, and `/page3`.

Q3. How many requests did each page get?

```
➜  os-lab3 cut -d' ' -f5 access.log | sort | uniq -c
    125 /page0
    125 /page1
    125 /page2
    125 /page3
```

Verification: Each of the 4 pages received exactly 125 requests. The sum ($125 \times 4 = 500$) accounts for all entries in the log.

Q4. Which user or users have the most failed requests, and how many?

```
➜  os-lab3 grep 'FAIL' access.log | cut -d' ' -f3 | sort | uniq -c | sort -rn | head
    -2
     15 user5
     15 user3
```

Result: `user5` and `user3` tied for the most failed requests, with **15 failures each**. Inspecting all users reveals:

```
➜  os-lab3 grep 'FAIL' access.log | cut -d' ' -f3 | sort | uniq -c | sort -rn
     15 user5
     15 user3
     14 user6
     14 user4
     14 user2
     14 user1
     14 user0
```

Sum: $15 + 15 + (14 \times 5) = 100$, confirming consistency with Q1.

Q5. How many requests did user3 make, and how many of them failed?

```
➜  os-lab3 grep 'user3 ' access.log | cut -d' ' -f4 | sort | uniq -c
     15 FAIL
     57 OK
```

Result: `user3` made a total of **72 requests** ($57 + 15$), of which **15 failed**.

Q6. Print the last 3 failed requests, showing only time and user

```
➜  os-lab3 grep 'FAIL' access.log | tail -3 | cut -d' ' -f2,3
10:10 user0
10:15 user5
10:20 user3
```

Result: The last three failed requests occurred at 10:10 (user0), 10:15 (user5), and 10:20 (user3).

Q7. Which login shells appear in /etc/passwd, and how many accounts use each?

```
➜  os-lab3 cut -d: -f7 /etc/passwd | sort | uniq -c | sort -rn
     33 /usr/bin/nologin
      2 /usr/bin/bash
      1 /usr/bin/zsh
```

```
      1 /usr/bin/git-shell
      1 /bin/bash
```

Result: The majority (33) are system service accounts assigned `/usr/bin/nologin`, followed by active user shells (`bash` and `zsh`).

Q8. Predict, then explain: why do ls /etc — wc -l and ls -l /etc — wc -l differ by exactly one?

```
➜  os-lab3 ls /etc | wc -l && ls -l /etc | wc -l
214
215
```

Explanation: When `ls` writes to a pipe, it outputs directory entries one per line with no header. In contrast, `ls -l` produces a long listing that begins with an initial summary header on line 1 stating the total allocated disk blocks (e.g., `total 1.1M`), followed by the directory entries. This summary header accounts for the extra line.

Bonus (+1): Failed Requests Where the Minute Ends in 0

```
➜  os-lab3 grep -E '10:[0-9]0 .* FAIL' access.log | head -5
2026-10-05 10:10 user3 FAIL /page2
2026-10-05 10:20 user6 FAIL /page0
2026-10-05 10:30 user2 FAIL /page2
2026-10-05 10:40 user5 FAIL /page0
2026-10-05 10:50 user1 FAIL /page2
➜  os-lab3 grep -E '10:[0-9]0 .* FAIL' access.log | wc -l
50
```

Result: There are exactly **50 failed requests** whose minute ends in 0.

Complete oneliners.sh Script

```
#!/bin/bash
# Q1. How many requests failed?
grep -c 'FAIL' access.log

# Q2. How many different pages were requested, and which?
cut -d' ' -f5 access.log | sort -u

# Q3. How many requests did each page get?
cut -d' ' -f5 access.log | sort | uniq -c

# Q4. Which user or users have the most failed requests, and how many?
grep 'FAIL' access.log | cut -d' ' -f3 | sort | uniq -c | sort -rn | head -2

# Q5. How many requests did user3 make, and how many of them failed?
grep 'user3 ' access.log | cut -d' ' -f4 | sort | uniq -c

# Q6. Print the last 3 failed requests, showing only time and user.
grep 'FAIL' access.log | tail -3 | cut -d' ' -f2,3

# Q7. Which login shells appear in /etc/passwd, and how many accounts use each? (
    field 7, separator :)
cut -d: -f7 /etc/passwd | sort | uniq -c | sort -rn
```

```
# Q8. PREDICT, then explain: why do ls /etc | wc -l and ls -l /etc | wc -l differ by
    exactly one?
ls /etc | wc -l && ls -l /etc | wc -l

# Bonus (+1): with grep -E, print only the lines where the minute ends in 0 and
    request failed
grep -E '10:[0-9]0 .* FAIL' access.log
```

1.6 Part 5. Your first script

1.6.1 Completed report.sh Script

```
#!/bin/bash
# usage: ./report.sh access.log
log=$1
if [ ! -f "$log" ]; then
    echo "no such file: $log" >&2
    exit 1
fi
total=$(wc -l < "$log")
fails=$(grep -c FAIL "$log")
echo "$total requests, $fails failed"

# Complete the TODO: loop for each of user0 ... user6
for u in {0..6}; do
    user="user$u"
    count=$(grep "$user FAIL" "$log" | wc -l)
    echo "$user: $count failures"
done
```

1.6.2 Execution and Validation

```
➜  os-lab3 chmod +x report.sh
➜  os-lab3 ./report.sh access.log
500 requests, 100 failed
user0: 14 failures
user1: 14 failures
user2: 14 failures
user3: 15 failures
user4: 14 failures
user5: 15 failures
user6: 14 failures
➜  os-lab3 ./report.sh nope.log ; echo "exit code: $?"
no such file: nope.log
exit code: 1
```

Bonus (+1): Scheduled Execution via cron

A cron entry was configured to automate execution every minute:

```
* * * * * /home/ionut/os-lab3/report.sh /home/ionut/os-lab3/access.log >> /home/ionut
    /os-lab3/report.log 2>&1
```

After waiting two minutes, inspecting the log verified repeated scheduled execution:

```
➜  os-lab3 cat ~/os-lab3/report.log
500 requests, 100 failed
user0: 14 failures
user1: 14 failures
user2: 14 failures
user3: 15 failures
user4: 14 failures
user5: 15 failures
user6: 14 failures
500 requests, 100 failed
user0: 14 failures
...
```

The crontab entry was subsequently cleaned up using `crontab -r`.

Observe:

1. **What does the first line #!/bin/bash do, and what happens if you run ./report.sh before chmod +x?**

   The first line `#!/bin/bash` is the **shebang**. When a script is executed via the `execve` system call, the Linux kernel detects the magic byte sequence `0x23 0x21` (`#!`), parses the remainder of the line as the path to the interpreter binary, and executes `/bin/bash` with the script file path as its argument.
   If you attempt to run `./report.sh` before `chmod +x`, the kernel checks the inode permission bits, detects that execute permission (`x`) is absent, and rejects execution with:

   ```
   bash: ./report.sh: Permission denied
   ```

2. **Why must you type ./report.sh and not just report.sh?**

   By default in Unix-like operating systems, the current working directory (`.`) is deliberately omitted from `$PATH` for security. If the shell searched the current directory by default, an unprivileged user could place a malicious script named `ls` or `cd` in a shared directory (like `/tmp`) and trick another user into executing it. Therefore, to run a program residing in the current directory, the relative path prefix `./` must be supplied explicitly.

3. **Why does the script write the error with ¿&2 and end with exit 1?**

   In standard Unix architecture, programs distinguish between standard data output (`stdout`, fd 1) and diagnostic/error telemetry (`stderr`, fd 2). Writing error messages with `>&2` ensures that if the caller redirects stdout into a pipeline or file, the error message still displays on the operator's terminal rather than contaminating the downstream data stream.
   Ending with `exit 1` signals to calling processes, parent shell scripts, or CI/CD pipelines that the program encountered a failure condition (non-zero exit code), triggering short-circuit evaluation operators such as `&&` and `||`.

2 Conclusion

In this laboratory work, I explored the practical mechanisms and operating principles of the Unix command line, focusing on shell mechanics, stream manipulation, modular text filters, and process control.

Through hands-on experimentation with `bash`, I examined how the shell parses command lines before dispatching system calls to the kernel. I analyzed the crucial distinctions between shell built-ins (such as `cd`, which directly mutates parent process context) and external executables located via `$PATH`. By constructing prediction tables, I traced the lifecycle of shell expansions: brace generation, tilde and variable interpolation, command substitution, word splitting, and pathname globbing. The experiments demonstrated that user-space programs remain completely oblivious to patterns like `*.c`; it is the shell that resolves wildcards against the directory tree and delivers finalized string arrays through `argv`.

I investigated the kernel's stream redirection architecture, observing how file descriptors 0, 1, and 2 can be independently mapped, duplicated via `dup2`, or redirected to pseudodevices like `/dev/null`. By chaining elementary Unix filters—`grep`, `cut`, `sort`, `uniq`, `wc`, `head`, and `tail`—into composable one-liner pipelines, I solved realistic analytics queries on web server access logs with mathematical precision. Finally, I authored and automated a robust shell script incorporating argument validation, error channel separation, and cron scheduling.

These exercises reinforced the Unix philosophy: building simple, single-purpose tools that interact seamlessly over universal byte-stream interfaces.